Laborator 10 - Static Checking

1. Objectives

Verificarea automată a calității codului reprezintă unul dintre primii pași în garantarea corectitudinii codului. Așa cum ne putem imagina, un program formatat inconsistent și cu greșeli frecvente de structură e predispus la a avea probleme ulterioare, chit că inițial el e funcțional. De asemenea, mai ales în proiectele open-source, ne dorim ca programele noastre să fie cât mai prezentabile pentru a atrage contribuitori și utilizatori, cât și pentru a face contribuțiile mai consistente. Dacă fiecare contribuitor ar avea un stil de formatare al codului diferit, s-ar ajunge garantat la neînțelegeri.

Aceste idei sunt întărite când este vorba de limbaje interpretate precum Python. După cum știm, în Python tipul variabilelor este determinat la rulare, deci nu putem ști cu siguranță că avem un cod funcțional, până când nu rulăm cu succes pe toate căile posibile de execuție. Totodată, codul Python este bazat pe indentare și o formatare cât de cât corectă, deci fără să respectăm un minim de formatare, este posibil ca interpretorul de Python să refuze sa ne ruleze programul cu totul. Deoarece Python este mai lax în scriere, trebuie să folosim folosim aplicații terțe pentru a ne formata codul, comparat cu limbaje mult mai stricte ca Rust.



Dacă combinăm toate aceste idei cu o formatare automată și folosirea sistemelor de CI/CD, precum în laboratoarele precedente, putem să ne asigurăm că în proiectul nostru nu pătrunde cod de o calitate inferioară celei dorite.

1.1 Linters

Linterele, sau uneltele de verificare a calității codului, reprezintă prima metodă de verificare a codului. De multe ori acestea sunt integrate în editorul folosit de developeri, în așa fel încât codul este evidențiat, sau chiar formatat automat la fiecare salvare a fișierului.

Deoarece procesul de linting poate semnala multe probleme, este recomandat să fie integrat în dezvoltarea unui program cât mai devreme, pentru a evita schimbări fundamentale ale codului ulterior. Dificultatea principală în procesul de linting vine din stabilirea unui standard de formatare a codului care să corespundă cu așteptările tuturor contribuitorilor. În cazul nostru, configurările de bază sunt mai mult decât suficiente.

1.2 Static Checking

Static checkerele, sau unelte de verificare statică a codului, reprezintă o verificare mai amănunțită a codului. Acestea parcurg codul și încearcă să îl înțeleagă în amănunt pentru a deduce folosiri eronate ale anumitor funcții și/sau cod. Un bun exemplu de astfel de verificări sunt cele de securitate. Prin anumite static checkere putem descoperi automat parole, tokene, și alte date de autentificare uitate în cod, înainte de a face public codul și a ne compromite. Totodată, vom vedea că există și unelte mai amănunțite, ce realizează o detecție automată a tipurilor fiecărei variabile, pentru a ne garanta că nu există erori de runtime generate de acestea.

Static checkerele reprezintă, pe scurt, următorul pas al verificării codului de către developer, înainte de rulare și de publicarea efectivă a codului.

2. Exercises

În exercițiile din laborator vom explora cinci unelte de linting și static checking. Pentru fiecare va trebui să o rulați, să rezolvați (pe cât se poate) problemele semnalate, și să realizați un commit descriptiv în legătură cu schimbările. Vom folosi un repository [https://github.com/Ingineria-Calculatoarelor-ACS-UPB/python-mini-project.git] open source pentru a rula exercițiile. Din acesta va trebui să alegeți 2-3 mini-proiecte pe care să rulați uneltele și să rezolvați problemele.

Dacă proiectul ales nu mai are destule probleme pentru a fi util, sau ați rezolvat deja toate problemele, puteți încerca altul.

2.0 Environment Setup

1. Instalăm pip dacă lipsește: https://pip.pypa.io/en/stable/installation
[https://pip.pypa.io/en/stable/installation]

```
$ python3 -m ensurepip --upgrade
```

2. Pentru început va trebui să instalăm tool-urile ce le vom folosi:

```
$ pip3 install pylint ruff black bandit pytype
```

3. Apoi clonăm repository-ul peste care o sa rulăm uneltele:

```
$ git clone https://github.com/Ingineria-Calculatoarelor-ACS-UPB/python-mini-project.git
```

Rulați comenzile din timp, eventual la început de laborator, deoarece instalarea poate să dureze.

2.1 Pylint - Basic Checks

Pylint [https://github.com/PyCQA/pylint] este unul dintre cele mai cunoscute unelte de linting. Acesta oferă informații simple, cât și avansate în legătură cu codul pe care îl analizează, și apoi generează un raport cu toate erorile și acordă o notă acestuia. Acesta are și integrare cu VSCode, precum și alte editoare, ce ușurează folosirea lui.

```
everynoise.py:36:0: C0301: Line too long (110/100) (line-too-long)
everynoise.py:1:0: C0114: Missing module docstring (missing-module-docstring)
everynoise.py:4:0: C0116: Missing function or method docstring (missing-function-docstring)
everynoise.py:5:15: W3101: Missing timeout argument for method 'requests.get' can cause your program to hang indefinitely (missing-ti
meout)
everynoise.py:9:0: C0116: Missing function or method docstring (missing-function-docstring)
everynoise.py:23:0: C0116: Missing function or method docstring (missing-function-docstring)
everynoise.py:33:0: C0116: Missing function or method docstring (missing-function-docstring)
everynoise.py:40:0: C0116: Missing function or method docstring (missing-function-docstring)
everynoise.py:51:0: C0116: Missing function or method docstring (missing-function-docstring)
everynoise.py:51:15: W0622: Redefining built-in 'list' (redefined-builtin)
everynoise.py:52:4: C0200: Consider using enumerate instead of iterating with range and len (consider-using-enumerate)
everynoise.py:58:0: C0116: Missing function or method docstring (missing-function-docstring)
everynoise.py:67:0: C0116: Missing function or method docstring (missing-function-docstring)
everynoise.py:74:0: C0116: Missing function or method docstring (missing-function-docstring)

-----------------------------------
Your code has been rated at 7.91/10
```

Observăm că fiecare linie din raport menționează un cod de verificare, prin care e identificată fiecare problemă. Putem avea probleme de tip error, warning, comment, etc.

1. Rulați Pylint și observați problemele semnalate de acesta.
2. Rezolvați problemele semnalate pentru proiectul ales.
3. Realizați un commit prin care să salvați modificările aduse.

2.2 Ruff - Advanced Checks

De cele mai multe ori, un singur linter nu poate prinde toate erorile ce le avem în cod, de aceea suntem nevoiți să folosim mai multe. Ruff [https://github.com/charliermarsh/ruff] este o suită de verificatoare de cod ce este des folosită în proiectele mari. Aceasta ne poate ajuta să descoperim probleme ce au fost ratate de Pylint. De asemenea, înglobează verificări de la diferite alte unelte.

```
everynoise.py:4:1: E302 expected 2 blank lines, found 1
everynoise.py:9:1: E302 expected 2 blank lines, found 1
everynoise.py:23:1: E302 expected 2 blank lines, found 1
everynoise.py:33:1: E302 expected 2 blank lines, found 1
everynoise.py:36:80: E501 line too long (110 > 79 characters)
everynoise.py:40:1: E302 expected 2 blank lines, found 1
everynoise.py:51:1: E302 expected 2 blank lines, found 1
everynoise.py:58:1: E302 expected 2 blank lines, found 1
everynoise.py:67:1: E302 expected 2 blank lines, found 1
everynoise.py:74:1: E302 expected 2 blank lines, found 1
everynoise.py:75:80: E501 line too long (84 > 79 characters)
everynoise.py:85:1: E305 expected 2 blank lines after class or function definition, found 1
```

1. Rulați Ruff și observați problemele semnalate de acesta.
2. Rezolvați problemele semnalate pentru proiectul ales.
3. Realizați un commit prin care să salvați modificările aduse.

2.3 Black - Automatic Checks

Am observat că este destul de obositor să facem atatea modificări și timpul consumat pe formatarea codului ar fi putut să fie investit în adăugarea de funcționalități noi. Black [https://github.com/psf/black] vine cu o paradigmă nouă, cea de cedare a controlului asupra linting-ului către linter. Pe lângă semnalarea problemelor, acesta le și rezolvă după un standard propus de dezvoltatori. Desigur, noi putem să rulăm fără a aplica modificările.

```
--- everynoise.py       2023-03-18 12:52:26.868457 +0000
+++ everynoise.py       2023-03-18 12:52:33.776848 +0000
@@ -1,42 +1,51 @@
 import requests
+

 # Download the html link
 def download_page(url):
     response = requests.get(url)
     return response.text


+
 # Extract only the artists from the html
 def parse_artists_page(page):
     lines = []
     for line in page.splitlines():
-        if line.startswith("<a href=\"engenremap"):
-            while line.find('target=_parent>') != -1:
-                start = line.find('target=_parent>') + 15
+        if line.startswith('<a href="engenremap'):
+            while line.find("target=_parent>") != -1:
+                start = line.find("target=_parent>") + 15
                 line = line[start:]
-                end = line.find('</a>')
+                end = line.find("</a>")
                 lines.append(line[:end])
                 line = line[end:]
             break
     return lines
```

```
reformatted everynoise.py

All done! ✨ 🍰 ✨
1 file reformatted.
```

1. Rulați Black în mod de diff color precizând argumentele --diff și --color la rulare și observați problemele.
2. Rulați Black fără argumente adiționale pentru a aplica schimbările.
3. Rezolvați problemele rămase semnalate pentru proiectul ales (dacă mai există).
4. Realizați un commit prin care să salvați modificările aduse.

2.4 Bandit - Security Checks

Bandit [https://github.com/PyCQA/bandit] este o unealtă de static checking concentrată pe partea de securitate. Aceasta se asigură că programul nostru nu poate fi exploatat de alte persoane și că este sigur în timpul folosirii. Pe lângă verificarea existenței unor parole/tokene uitate, acesta analizează și cod ce ar putea fi exploatat în anumite circumstanțe.

```
Run started:2023-03-18 10:46:24.237814

Test results:
>> Issue: [B113:request_without_timeout] Requests call without timeout
   Severity: Medium   Confidence: Low
   CWE: CWE-400 (https://cwe.mitre.org/data/definitions/400.html)
   More Info: https://bandit.readthedocs.io/en/1.7.5/plugins/b113_request_without_timeout.html
   Location: everynoise.py:6:15
5       def download_page(url):
6           response = requests.get(url)
7           return response.text

--------------------------------------------------

Code scanned:
        Total lines of code: 72
        Total lines skipped (#nosec): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 0
                Medium: 1
                High: 0
        Total issues (by confidence):
                Undefined: 0
                Low: 1
                Medium: 0
                High: 0
Files skipped (0):
```

În cazul de față, avem o singură problemă, aceea că descărcăm o pagină fără a menționa un timp de oprire. Un utilizator malițios poate să apeleze funcția cu un url invalid și să blocheze execuția programului.

1. Rulați Bandit și observați problemele.
2. Rezolvați problemele semnalate pentru proiectul ales.
3. Realizați un commit prin care să salvați modificările aduse.

2.5 Pytype - Type Checks

Pytype [https://github.com/google/pytype] este un tool de static checking mai avansat față de cele precedente. Se folosește de un engine de analiză statică pentru a garanta corectitudinea tipurilor variabilelor din codul nostru. Acesta ideal este folosit în timpul dezvoltării programului, sau atunci când avem bug-uri. Este foarte probabil ca acesta să nu întoarcă nimic, în cazul proiectelor analizate în cadrul laboratorului, deoarece acestea sunt deja funcționale.

```python
def f():
    return "PyCon"
def g():
    return f() + 2019

# pytype: line 4, in g: unsupported operand type(s) for +: 'str'
# and 'int' [unsupported-operands]
```

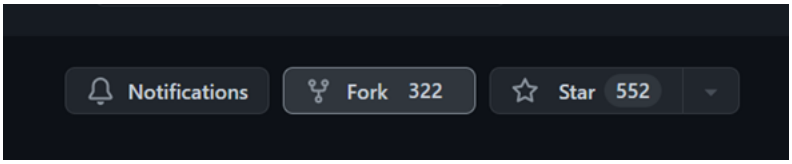
Fără a rula codul, acesta a analizat operațiile realizate și a detectat adunarea unui întreg cu un șir de caractere. În Python astfel de problemă ar fi apărut doar la rulare, și dacă nu am testat acel caz posibil să treacă nedetectată și să ajungă în mediu de producție.

```
Computing dependencies
/usr/local/lib/python3.7/dist-packages/urllib3/contrib/
tion of optional dependencies: specifically, PySocks.
l#socks-proxies
  DependencyWarning,
Analyzing 1 sources with 0 local dependencies
ninja: Entering directory `.pytype'
[1/1] check everynoise
Leaving directory '.pytype'
Success: no errors found
```

1. Rulați Pytype și observați problemele.
2. Rezolvați problemele semnalate pentru proiectul ales (dacă există).
3. Realizați un commit prin care să salvați modificările aduse.

3. Bonus

Pentru a realiza bonusul va trebui să creăm și un fork în care să adăugăm pipeline-urile:



3.1 CI Pipeline

În fork-ul realizat în partea de setup, adăugați un fișier **python-checks.yaml** în calea **.github/workflows** și completați conform specificației următoare:

- Verificarea trebuie să se realizeze pe tot codul.
- Verificarea trebuie sa pornească atunci când este deschis un nou Pull Request.
- Workflow-ul trebuie să aibă 5 pași, corespunzător fiecărei unelte: Pylint, Ruff, Black, Bandit, Pytype. (Black poate sa aplice automat modificările).
- Workflow-ul trebuie să eșueze atunci când oricare dintre cele 5 tool-uri a detectat erori, dar să își continue execuția.
- Workflow-ul este complet doar atunci când niciun tool nu a semnalat erori.

Aveți grijă să rulați uneltele în mod neinteractiv (dacă este disponibil), și să verificați codurile de eroare întoarse de acestea.

4. Resources

- https://www.spiceworks.com/tech/devops/articles/what-is-ci-cd/ [https://www.spiceworks.com/tech/devops/articles/what-is-ci-cd/]
- https://github.com/google/pytype [https://github.com/google/pytype]
- https://github.com/PyCQA/bandit [https://github.com/PyCQA/bandit]
- https://github.com/psf/black [https://github.com/psf/black]
- https://github.com/charliermarsh/ruff [https://github.com/charliermarsh/ruff]
- https://github.com/PyCQA/pylint [https://github.com/PyCQA/pylint]
- ic_lab10_demo_everynoise.zip

tsc/laboratoare/laborator-10.txt · Last modified: 2024/02/29 14:40 (external edit)